# Google Merchant Centre Task

Combined task of Oauth flow plus fetching merchant account list

## Task Description:

Create Sign in with google workflow via Oauth integration, implementing the logic of fetching the access and refresh token to access other google api. Automatically the access token should be refreshed on expiry. After successful login, fetch the available merchant account list and show it to the user. Also provide the option to select the desired account. Handle proper authentication and authorization, persistent storage using Database and appropriate workflow and security.

## Technologies Used:

- Fastapi
- PyMongo
- Google Shopping Merchants Framework
- Authlib
- Python
- HTML (Jinja2 Templates)
- Docker
- Git

## Folder Structure:

```
/
    /app
        /config
            __init.py
            db.py
            settings.py
        /error
            __init__.py
            exceptions.py
        /log
            __init__.py
            logger.py
        /routes
            __init__.py
            auth.py
            merchant.py
        /templates
            dashboard.html
```

```
                    home.html
                    merchant.html
                    404_error.html
            /utilities
                    __init__.py
                    auth_utils.py
                    merchant_utils.py
            __init.py
            main.py
            .env
    .dockerignore
    .gitignore
    Dockerfile
    docker-compose.yaml
    requirements.txt
```

# Steps to Run the project:

1. On Local Machine
    a. git clone the project root directory
    b. Create python virtual environment using “python3 -m venv venv”
    c. Activate the virtual environment using “source venv/bin/activate”
    d. Install the dependencies using “pip install -r requirements.txt”
    e. Configure the .env file
    f. Run the project from the root directory using “uvicorn app.main:app”

2. Using Docker
    a. git clone the project root directory
    b. Run the command “docker compose up”

## .env file format:

```
GOOGLE_CLIENT_ID=”<YOUR-GOOGLE-PROJECT-CLIENT-ID>”
GOOGLE_CLIENT_SECRET=<”YOUR-GOOGLE-PROJECT-CLIENT-SECRET>”
SESSION_SECRET_KEY=”<ANY-ARBITARY-SESSION-SECRET-KEY>”
SECRET_KEY=”<ANY-ARBITARY-SECRET-KEY>”
MONGO_URI=”<YOUR-MONGODB-URI>”
```

# Api Endpoints:

- **/** -> Shows the user dashboard 🛡️
- ***/home*** -> Home/Landing page
- ***/auth/google/login*** -> Login with Google Endpoint

- ***/auth/google/callback*** -> Callback URL configured in the cloud console project
- ***/auth/change-account*** -> Change the current google account and switch to another one 🛡️
- ***/auth/hard-logout*** -> Revoke the access to the account credentials along with session credentials 🛡️
- ***/auth/soft-logout*** -> Revoke the access to the session credentials only
- ***/merchants*** -> Fetch all list of merchants associated with the account 🛡️
- ***/merchants/select-merchant*** -> Select the merchant-id to be used 🛡️

## Database Schema:

- users
  - _id (ObjectId)
  - name (string)
  - email (string)
  - method (string)
- oauth_tokens
  - _id (ObjectId)
  - user (string)
  - access_token (string)
  - refresh_token (string)
  - expires_at (float)
- merchant_accounts
  - _id (ObjectId)
  - user_id (string)
  - google_account (string)
  - merchants (array)
  - selected_merchant_id (integer)

## Implementation Approach:

Oauth was implemented via the authlib library. Currently google oauth library lacks the asynchronous functionality leading to preference of authlib over google-oauth. Firstly, when the api endpoint /auth/google/login is hit it is redirected to the google's authorization url to obtain the code being used in exchange of the tokens. After usual sign in process from the user end, it is redirected to the /auth/google/callback (also configured on the google cloud console project) which in turn call the token url to obtain the access token (short lived) and refresh token (long lived). After successfully fetching both the tokens, they are stored in the mongodb collection oauth_tokens under the logged in user id along with the expiry timestamp of the access token. Also to manage the sessions, custom jwt access and refresh tokens are created which are stored in fastapi session. All the routes marked as 🛡️ are authenticated routes i.e. the presence of valid access token is must for the user session. Revocation of the google provided tokens and refreshing the access token on expiry is carried out via explicitly calling the respected url using the httpx library in python. Though on the user end, for refreshing the access token is

carried out implicitly via the code itself not prompting the user again for authentication and carrying the oauth flow.

Merchants are fetched using google-shopping-merchants-v1 library which provides methods and classes to fetch the merchant accounts directly via the access token. The fetched merchant list is then stored in the mongodb collection merchant_accounts and if no account is selected by default the first fetched account is chosen. The merchants/select-account route chooses the particular merchant-account and stores it in the mongodb selected_merchant_id field.

***Read the code Docstrings for more details…***

## Demonstration Link:

**https://drive.google.com/file/d/156O_jFlHb4ttUKkbcmu5PiYglXdGp-jE/view?usp=sharing**